```
class FlightModel
end
```

that can be overridden by each airplane, for instance

```
# app/models/bell_x1/flight_model.rb
module BellX1
  class FlightModel < FlightModel
  end
end

# app/models/bell_x1/aircraft.rb
module BellX1
  class Aircraft
    def initialize
      @flight_model = FlightModel.new
    end
  end
end
```

The initializer wants to create a `BellX1::FlightModel` and nesting has `BellX1`, that looks good. But if the default flight model is loaded and the one for the Bell-X1 is not, the interpreter is able to resolve the top-level `FlightModel` and autoloading is thus not triggered for `BellX1::FlightModel`.
That code depends on the execution path.

These kind of ambiguities can often be resolved using qualified constants:

```
module BellX1
  class Plane
    def flight_model
      @flight_model ||= BellX1::FlightModel.new
    end
  end
end
```

Also, `require_dependency` is a solution:

```
require_dependency 'bell_x1/flight_model'

module BellX1
  class Plane
    def flight_model
      @flight_model ||= FlightModel.new
    end
  end
end
```

10.6.2 Qualified References

Given

```
# app/models/hotel.rb
class Hotel
end

# app/models/image.rb
class Image
end

# app/models/hotel/image.rb
class Hotel
  class Image < Image
  end
end
```